

Every question on this assignment is worth 10 points.

Q1. Prove that a set is semi-decidable iff it is m -reducible to $H = \{(e, x) \in \mathbb{N}^2 : M_e(x) \text{ halts}\}$.

Solution

Proof. Recall the relevant definitions. A set $A \subseteq \mathbb{N}$ is *semi-decidable* if there exists a Turing machine that halts on input x exactly when $x \in A$. A set A is *many-one reducible* to a set B (written $A \leq_m B$) if there exists a total computable function f such that

$$x \in A \iff f(x) \in B.$$

The halting set is $H = \{(e, x) \in \mathbb{N}^2 : M_e(x) \text{ halts}\}$.

We show A is semi-decidable $\iff A \leq_m H$.

($\Rightarrow$) If A is semi-decidable, then $A \leq_m H$.

Assume A is semi-decidable, so there is a Turing machine M such that M halts on input x if and only if $x \in A$.

Key Tool (s-m-n Theorem). For any fixed input x , we can effectively construct a program P_x that *ignores its own input* and simply simulates M on x . The s-m-n theorem guarantees that the index e_x of such a program P_x can be computed from x by a total computable function.

Using s-m-n, define the total computable function

$$f(x) = \langle e_x, 0 \rangle,$$

where $\langle \cdot, \cdot \rangle$ is a fixed computable pairing function and e_x is the index of P_x .

Machine P_x on input y (ignores y):

- (a) Simulate M on x .
- (b) If $M(x)$ halts, then halt.

Summary:

- If $x \in A$, then M halts on x , so P_x halts on input 0. Thus $(e_x, 0) \in H$, i.e. $f(x) \in H$.
- If $x \notin A$, then M does not halt on x , so P_x does not halt on input 0. Thus $(e_x, 0) \notin H$, i.e. $f(x) \notin H$.

Since f is total and computable (by s-m-n), we have $A \leq_m H$.

($\Leftarrow$) If $A \leq_m H$, then A is semi-decidable.

Assume there is a total computable function f such that $x \in A \iff f(x) \in H$. We first observe that H is itself semi-decidable:

Observation. H is semi-decidable via a universal simulator: on input (e, y) , simulate $M_e(y)$ step by step and halt if and only if that simulation halts.

We construct a semi-decider for A as follows.

Machine M_A on input x :

- (a) Compute $y := f(x)$ (possible since f is total computable).
- (b) Parse y as a pair (e', x') and simulate $M_{e'}(x')$ step by step.
- (c) If that simulation ever halts, then halt.

Summary:

- If $x \in A$, then $f(x) \in H$, so the simulation in step 2 eventually halts, and M_A halts.
- If $x \notin A$, then $f(x) \notin H$, so the simulation never halts, and M_A runs forever.

Thus M_A halts on input x exactly when $x \in A$, so A is semi-decidable.

A set is semi-decidable if and only if it is m -reducible to H .

□

Q2. Prove that every infinite semi-decidable set contains an infinite decidable subset.

Solution

Proof. Let $A \subseteq \mathbb{N}$ be infinite and semi-decidable. We construct an infinite decidable set $S \subseteq A$ explicitly.

Step 1: Enumerating A .

Since A is semi-decidable, there exists a computable enumerator (a Turing machine ENUM) that when run indefinitely, outputs all elements of A (possibly with repetitions).

Step 2: Constructing a computable strictly increasing sequence.

We define a total computable function $f : \mathbb{N} \rightarrow \mathbb{N}$ by recursion, producing a strictly increasing sequence $f(0) < f(1) < f(2) < \dots$ of elements of A .

Definition of f by recursion:

- (a) **Base case.** Run ENUM until it outputs some element $a \in A$. Set $f(0) := a$.
- (b) **Recursive case.** Given $f(n)$, run ENUM and wait until it outputs some element strictly greater than $f(n)$. Set that element to be $f(n+1)$.

Why f is total. At each stage, we need ENUM to eventually produce a value greater than $f(n)$. Since A is infinite, only finitely many elements of A are $\leq f(n)$, so infinitely many elements of A exceed $f(n)$. Since ENUM lists every element of A , one of these larger elements will appear in finite time. Hence the search always terminates and f is a total computable function.

Step 3: Defining S and verifying $S \subseteq A$ is infinite.

Let $S = \{f(0), f(1), f(2), \dots\}$ be the range of f . By construction, every $f(n) \in A$, so $S \subseteq A$. Since the sequence is strictly increasing, all values are distinct, so S is infinite.

Step 4: S is decidable.

We describe a Turing machine M_S that decides membership in S on any input $y \in \mathbb{N}$.

Machine M_S on input y :

- (a) Generate $f(0), f(1), f(2), \dots$ one value at a time.
- (b) At each stage k , compare $f(k)$ with y :
 - If $f(k) = y$: **accept** (i.e., $y \in S$).
 - If $f(k) > y$: **reject** (i.e., $y \notin S$).

Summary and termination:

- If $y \in S$, then $y = f(k)$ for some k , and M_S accepts at stage k .
- If $y \notin S$, then since f is strictly increasing and unbounded, eventually some $f(k) > y$ is reached, and M_S rejects at that stage.

In both cases M_S halts in finite time, so S is decidable.

$S = \{f(0), f(1), f(2), \dots\} \subseteq A$ is an infinite decidable subset of A .

□

Q3. Prove that if a set A of natural numbers is semi-decidable and infinite, then there exists an injective computable function $f: \mathbb{N} \rightarrow \mathbb{N}$ such that $\text{Range}(f) = A$.

Solution

Proof. Since A is semi-decidable, there is a computable enumerator ENUM that outputs a sequence

$$e_0, e_1, e_2, \dots$$

such that every element of A appears at least once among the e_i (repetitions are allowed). Since A is infinite, infinitely many of the e_i are distinct.

Construction of f .

We define $f(n)$ to be the $(n+1)$ -th *distinct* value that appears in the enumeration. Concretely:

Machine M_f on input n :

- (a) Initialise **seen** $:= []$ and $i := 0$.
- (b) Repeat:
 - Compute $a := e_i$ (the i -th output of ENUM).
 - If a does not appear in **seen**, append a to **seen**.
 - Increment i .
- (c) Stop as soon as $|\text{seen}| = n + 1$. Return **seen** $[n]$.

f is total and computable.

Each step queries ENUM for one value, which is computable. The loop terminates because the enumeration contains infinitely many distinct elements of A ; hence for any fixed n , we will eventually accumulate $n + 1$ distinct values. Therefore f always halts and is a total computable function.

f is injective.

A value is added to **seen** only when it has not been seen before. Thus **seen** $[0]$, **seen** $[1]$, $\dots$ are pairwise distinct, which means $f(m) = f(n) \implies m = n$. Hence f is injective.

$\text{Range}(f) = A$.

- $\text{Range}(f) \subseteq A$. Every value appended to **seen** is an output e_i of ENUM , so each $f(n) \in A$.
- $A \subseteq \text{Range}(f)$. Every $a \in A$ appears at least once in the enumeration. The first time a appears it is appended to **seen** at some position k , giving $f(k) = a$. Hence every element of A is in the range of f .

Combining both inclusions gives $\text{Range}(f) = A$.

f is a total computable injective function with $\text{Range}(f) = A$.

□

- Q4. Many standard Gödel numbering schemes (such as those based on prime factorization or direct string encoding) are injective but not surjective – leaving “gaps” of natural numbers that do not correspond to valid Turing machine syntax. Prove that there exists a computable bijection between the set of Gödel numbers of (valid syntax) Turing machines and the natural numbers.

Solution

Proof. Let $G \subseteq \mathbb{N}$ denote the set of all Gödel numbers that encode syntactically valid Turing machines under a fixed standard encoding. The proof rests on two key observations.

Key Facts about G .

- (a) **G is decidable.** Given any $x \in \mathbb{N}$, one can effectively check whether x parses as a valid TM description (checking syntax is a finite, mechanical procedure). So there is a Turing machine that decides membership in G .
- (b) **G is infinite.** For each $n \in \mathbb{N}$ one can construct a valid TM with n additional dummy states, yielding infinitely many syntactically distinct (hence differently encoded) valid TMs.

Construction of the bijection.

Since G is decidable and infinite, we can scan the natural numbers in increasing order and list the elements of G , producing a strictly increasing sequence

$$g(0) < g(1) < g(2) < \dots$$

whose range is exactly G . Concretely, $g(n)$ is computed as follows.

Machine M_g on input n :

- (a) Set $i := 0$ and $count := 0$.
- (b) Repeat:
 - Run the decider for G on input i .
 - If $i \in G$: if $count = n$, **return** i ; otherwise increment $count$.
 - Increment i .

The decider for G always halts, and because G is infinite the loop finds the $(n + 1)$ -th valid code in finite time. Hence g is a *total computable* function.

$g : \mathbb{N} \rightarrow G$ is a computable bijection.

- *Injective.* The sequence is strictly increasing, so $g(m) = g(n) \implies m = n$.

- *Surjective.* Every $x \in G$ is eventually reached by the scan and is returned as $g(k)$ for some k , so $\text{Range}(g) = G$.

The inverse $g^{-1} : G \rightarrow \mathbb{N}$ is also computable.

Given $x \in G$, generate $g(0), g(1), \dots$ until some $g(k) = x$ and return k . Since g is strictly increasing, if $g(k) > x$ is reached we know x was never in G , but for $x \in G$ the search always terminates. Thus g^{-1} is computable on G .

$g : \mathbb{N} \rightarrow G, \quad n \mapsto g(n)$ is a computable bijection between $\mathbb{N}$ and G .

□

Q5. Prove that the set $\mathbf{Tot} = \{e \in \mathbb{N} : M_e \text{ has domain } \mathbb{N}\}$ (i.e. the set of indices of total computable functions) is not semi-decidable.

Your solution must assume towards a contradiction that **Tot** is semi-decidable. Then, show that this assumption implies the existence of some computable f such that $\{f(e) : e \in \mathbb{N}\}$ is the set of indices of computable functions. This will imply that the function given by $g(x) = M_{f(x)}(x) + 1$ is computable (explain why). Finally, find a contradiction.

Solution

Proof. Assume for contradiction that **Tot** is semi-decidable.

Step 1: Obtaining a computable enumeration of Tot.

If **Tot** is semi-decidable, then it has a computable enumerator: by dovetailing the semi-decision procedure over all indices and collecting those that are accepted, we obtain a total computable function

$$f : \mathbb{N} \rightarrow \mathbb{N}$$

whose range is exactly **Tot**. Fix such an f for the remainder of the proof.

Step 2: Defining g and showing it is total and computable.

Define

$$g(x) := M_{f(x)}(x) + 1.$$

Why g is total and computable. For each x , we have $f(x) \in \mathbf{Tot}$ (since f enumerates **Tot**), so $M_{f(x)}$ is a *total* function that halts on every input. The following procedure always terminates:

- (a) Compute the index $f(x)$ (possible since f is total computable).
- (b) Simulate $M_{f(x)}$ on input x (this always halts since $f(x) \in \mathbf{Tot}$).
- (c) Return the result plus 1.

Hence g is a total computable function.

Step 3: Applying the Recursion Theorem.

Kleene's Recursion Theorem. For every total computable function t , there exists an index e such that $M_e = M_{t(e)}$, i.e. the program at index e computes the same function as the program at index $t(e)$.

Since g is total and computable, by the Recursion Theorem there exists an index $e \in \mathbb{N}$ such that

$$M_e(y) = g(y) \quad \text{for every } y \in \mathbb{N}.$$

In particular, M_e is total (it computes g , which is total), so $e \in \mathbf{Tot}$.

Step 4: Deriving the contradiction.

Since f enumerates $\mathbf{Tot}$ and $e \in \mathbf{Tot}$, there exists some $k \in \mathbb{N}$ with $f(k) = e$. Evaluating g at k using both its definition and the fixed-point property gives

$$g(k) = M_{f(k)}(k) + 1 = M_e(k) + 1 = g(k) + 1,$$

which is a contradiction.

Tot is not semi-decidable.

□

Q6. Prove that the Predecessor function is primitive recursive.

Solution

Proof. Recall that the *predecessor* function is defined as

$$\text{pred}(n) = \begin{cases} 0 & n = 0, \\ n - 1 & n > 0. \end{cases}$$

Primitive Recursive Building Blocks. A function is primitive recursive if it is obtained from the *initial functions* (the zero function $Z() = 0$, the successor $S(x) = x+1$, and the projections $\pi_i^k(x_1, \dots, x_k) = x_i$) by finitely many applications of **composition** and **primitive recursion**.

Defining pred by primitive recursion.

We exhibit pred directly as a primitive recursion. Define the two ingredients:

- **Base case.** Let $g() = 0$, i.e. the zero function Z . This is an initial primitive recursive function.
- **Recursion step.** Let $h(u, v) = u = \pi_1^2(u, v)$, i.e. the first projection on two arguments. This is also an initial primitive recursive function.

Applying the primitive recursion scheme with g and h yields the unique function f satisfying

$$\begin{cases} f(0) = g() = 0, \\ f(n+1) = h(n, f(n)) = n. \end{cases}$$

Verification against the definition of pred:

- $f(0) = 0 = \text{pred}(0)$. ✓
- $f(n+1) = n = \text{pred}(n+1)$ for all $n \geq 0$. ✓

Thus $f = \text{pred}$. Since $g = Z$ and $h = \pi_1^2$ are both initial primitive recursive functions, and primitive recursion preserves primitive recursiveness, pred is primitive recursive.

$$\text{pred}(n) = \begin{cases} 0 & n = 0, \\ n - 1 & n > 0 \end{cases} \text{ is primitive recursive, via } g = Z \text{ and } h = \pi_1^2.$$

□

Q7. If $\mathcal{C} \subseteq \mathcal{P}(\mathbb{N})$, and $X \in \mathcal{C}$, we will say that X is $\mathcal{C}$ -complete if $\forall Y \in \mathcal{C}, Y \leq_m X$.

Prove that $\mathbf{Fin} = \{e \in \mathbb{N} : M_e \text{ halts on only finitely many elements}\}$ is Σ_2^0 -complete.

Solution

Proof. We must show two things: (1) $\mathbf{Fin} \in \Sigma_2^0$, and (2) every Σ_2^0 set many-one reduces to $\mathbf{Fin}$.

Part 1: $\mathbf{Fin} \in \Sigma_2^0$.

Let $T(e, x, s)$ be the decidable predicate “machine M_e halts on input x within s steps” (decidable by finite simulation). Then $e \in \mathbf{Fin}$ if and only if there is some bound k such that every input on which M_e halts is at most k . Formally:

$$e \in \mathbf{Fin} \iff \exists k \forall x \forall s (\neg T(e, x, s) \vee x \leq k).$$

This has the shape $\exists k \forall x P(e, k, x)$ with P decidable (since T is decidable). Therefore $\mathbf{Fin} \in \Sigma_2^0$.

Part 2: $\mathbf{Fin}$ is Σ_2^0 -hard.

Take any Σ_2^0 set A . By definition there is a decidable relation $R(x, y, z)$ such that

$$x \in A \iff \exists y \forall z R(x, y, z).$$

We construct a total computable function f with $x \in A \iff f(x) \in \mathbf{Fin}$.

Key Idea. For each x , we build a machine $M_{f(x)}$ whose domain is *finite* if a witness y with $\forall z R(x, y, z)$ exists, and *infinite* otherwise. We do this by constructing a computable enumerator for the domain that permanently stops as soon as it finds such a witness.

Construction of the enumerator for W_x . Fix x . At each stage $s = 0, 1, 2, \dots$:

Stage s :

- (a) Search for a $y \leq s$ such that $\forall z \leq s, R(x, y, z)$. (This check is finite and decidable.)
- (b) If such a y is found: **stop the enumeration forever** and enumerate nothing at this or any later stage.
- (c) If no such y is found: enumerate s into W_x .

Summary:

- If $x \in A$, let y^* be a witness with $\forall z R(x, y^*, z)$. For all sufficiently large s we have $y^* \leq s$ and $\forall z \leq s, R(x, y^*, z)$, so the procedure finds y^* at the first such stage and permanently stops enumerating. Only finitely many numbers were added to W_x , so W_x is *finite*.
- If $x \notin A$, then every candidate y has some counterexample z , which will eventually satisfy $z \leq s$. Hence no stopping candidate is ever found, and s is enumerated at every stage s . So W_x is *infinite*.

Obtaining the index $f(x)$. By the s-m-n theorem, from the enumerator above we can effectively compute an index $f(x)$ such that $\text{domain}(M_{f(x)}) = W_x$. Concretely, $M_{f(x)}$ on input n simulates the enumerator until n is enumerated (and halts), or runs forever if n is never enumerated. The map $x \mapsto f(x)$ is total and computable.

Therefore:

$$x \in A \iff W_x \text{ is finite} \iff \text{domain}(M_{f(x)}) \text{ is finite} \iff f(x) \in \mathbf{Fin}.$$

So f is a computable many-one reduction from the arbitrary Σ_2^0 set A to **Fin**, and hence **Fin** is Σ_2^0 -hard.

Fin $\in \Sigma_2^0$ and every Σ_2^0 set reduces to **Fin**, so **Fin** is Σ_2^0 -complete.

□

Q8. For A, B sets of natural numbers, we will write $A \leq_p B$ if $A \leq_m B$ via a reduction function that is computable in polynomial time. More clearly, $A \leq_p B$ means that:

There exists $f: \mathbb{N} \rightarrow \mathbb{N}$ such that f is computable in polynomial time, and

$$\forall x \in \mathbb{N}, \quad x \in A \Leftrightarrow f(x) \in B.$$

This new reducibility is known as **polynomial-time** reduction, which is important in studying complexity classes.

Let $P = \{X \subseteq \mathbb{N}: X \text{ is decidable in polynomial time}\}$.

- (a) Let $U \in P$, and suppose that $X \leq_p U$. Prove that X must be in P .
- (b) Prove that $\forall Y \in P, Y \leq_p \{1\}$.
- (c) [True or False] $\forall Y \in \Sigma_2^0, Y \leq_p \mathbf{Fin}$. Please justify.
- (d) [True or False] $\forall Y \in \Sigma_1^0, Y \leq_p H$ (the halting set). Please justify.

Solution

- (a) Let $U \in P$, and suppose that $X \leq_p U$. Prove that X must be in P .

Proof. Since $X \leq_p U$, there is a polynomial-time computable function f with $x \in X \iff f(x) \in U$. Since $U \in P$, there is a polynomial-time decider D_U for U . We decide X as follows.

Decider for X on input x :

- i. Compute $y := f(x)$, taking time $p(|x|)$ for some polynomial p .
- ii. Run D_U on input y . If D_U accepts, accept; otherwise reject. This takes time $q(|y|)$ for some polynomial q .

Since a polynomial-time algorithm cannot produce an output longer than a polynomial in its input length, $|y|$ is bounded by some polynomial in $|x|$, so $q(|y|)$ is also bounded by a polynomial in $|x|$. The total running time is therefore polynomial in $|x|$, and the decider is correct since $x \in X \iff f(x) \in U$. Hence $X \in P$. $\square$

- (b) Prove that $\forall Y \in P, Y \leq_p \{1\}$.

Proof. Let $Y \in P$, so there is a polynomial-time decider D_Y for Y . Define

$$f(x) = \begin{cases} 1 & \text{if } x \in Y, \\ 0 & \text{if } x \notin Y. \end{cases}$$

Computing $f(x)$ amounts to running D_Y on x and outputting a single bit, which is clearly polynomial time. Moreover,

$$x \in Y \iff f(x) = 1 \iff f(x) \in \{1\}.$$

So f is a polynomial-time many-one reduction from Y to $\{1\}$, giving $Y \leq_p \{1\}$. $\square$

- (c) [True or False] $\forall Y \in \Sigma_2^0, Y \leq_p \mathbf{Fin}$. Please justify.

False.

Recall. $\mathbf{Fin}$ is Σ_2^0 -complete under *computable* many-one reductions. That is, every Σ_2^0 set reduces to $\mathbf{Fin}$ via a general computable function, but such reductions may require unbounded computation to produce the appropriate machine index $f(x)$.

Polynomial-time reductions impose a strict resource bound: $f(x)$ must be computable in time polynomial in $|x|$. The Σ_2^0 class contains undecidable sets of arbitrarily high computational complexity, and encoding membership in such sets into $\mathbf{Fin}$ via a single function f generally requires an unbounded search to produce the correct machine index. There is no guarantee, and in fact it is false in general, that this can be done in polynomial time. Hence $\forall Y \in \Sigma_2^0, Y \leq_p \mathbf{Fin}$ is false.

- (d) [True or False] $\forall Y \in \Sigma_1^0, Y \leq_p H$ (the halting set). Please justify.

False.

The reasoning is the same as in part (3). H is Σ_1^0 -complete under *computable* many-one reductions: the classical proof takes an arbitrary r.e. set Y and constructs, for each input x , the index of a machine whose halting behaviour encodes whether $x \in Y$. Producing that index may require arbitrarily long computation and is not in general doable in polynomial time. Since Σ_1^0 contains sets of arbitrarily high time complexity, no single polynomial-time computable f can reduce all of them to H . Hence the statement is false.

Summary for (3) and (4). The completeness of $\mathbf{Fin}$ (for Σ_2^0) and H (for Σ_1^0) is established using *computable* many-one reductions, which carry no resource bound. Polynomial-time reductions are strictly stronger, and the classical constructions that witness completeness cannot in general be performed in polynomial time.

Q9. Bonus

The last question part (b) shows that $\leq_p$ trivializes the concept of completeness for the class P , which is a very important class in Complexity Theory. In practice, P -completeness is based on another type of reduction known as **Log-Space** reduction (denoted $\leq_L$). We write $A \leq_L B$ if the reduction function is computable in $O(\log n)$ space (excluding the space for the read-only input and write-only output). Read about Log-space reduction, and the Circuit Value Problem (CVP).

Prove that: CVP is P -complete (here P -completeness is with respect to $\leq_L$).

Solution

Proof. We prove two things: (1) $\text{CVP} \in P$, and (2) every $L \in P$ log-space reduces to CVP.

Definition (CVP). An instance of the Circuit Value Problem is a Boolean circuit C (a DAG whose internal nodes are AND, OR, NOT gates and whose leaves are input gates labelled with Boolean values) together with an assignment to its inputs. The question is: does C evaluate to 1 on that assignment?

Part 1: $\text{CVP} \in P$.

Given a circuit C and an input assignment, topologically order the gates and evaluate each gate in order (every gate's inputs are already evaluated when we reach it). This takes time linear in the number of gates, hence polynomial in the input size. So CVP has a polynomial-time decider and $\text{CVP} \in P$.

Part 2: CVP is P -hard under $\leq_L$.

Let $L \in P$ be arbitrary, decided by a deterministic Turing machine M in time $p(n)$ for some polynomial p . We construct a log-space computable reduction r mapping each input x to a circuit instance $r(x) = \langle C_x, \text{assignment} \rangle$ such that

$$x \in L \iff C_x(\text{assignment}) = 1.$$

Circuit construction (simulating M by a circuit).

Model the computation of M on input x (of length n) as a time-by-space *tableau*: a grid of cells where cell (t, i) records the tape symbol, head position, and state at time step t and tape position i . The tableau has dimensions $p(n) \times p(n)$.

Circuit C_x structure:

- (a) **Input gates.** One gate per input bit of x , plus constant gates for the initial state and head position.
- (b) **Update gadgets.** For each cell $(t + 1, i)$, a small constant-size subcircuit computing the new value from cells $(t, i - 1)$, (t, i) , $(t, i + 1)$ (encoding M 's local transition function). These gadgets are constant size so the total circuit size is polynomial in $p(n)$.
- (c) **Output gate.** The single gate encoding whether M accepts at time $p(n)$.

Why the reduction $x \mapsto \langle C_x, \text{assignment} \rangle$ is log-space computable.

The circuit is extremely regular: every gate can be *indexed* by a pair (t, i) , and its type and fan-in connections are determined entirely by M 's fixed transition function and those indices. A log-space transducer can therefore enumerate and output the circuit description on the fly:

Log-space transducer. Loop over all gate indices (t, i) with $t \in \{0, \dots, p(n)\}$ and $i \in \{1, \dots, p(n)\}$. For each gate:

- (a) Store t and i in binary, where each counter uses $O(\log n)$ bits.
- (b) Compute the gate type and the indices of its fan-in gates by simple arithmetic on t and i (increment, decrement, add small constants). This is all doable in $O(\log n)$ space.
- (c) Print the gate description and advance the counters.

The transducer reads x on a read-only tape and writes the circuit to a write-only output tape; the only working space needed is $O(\log n)$ bits for the binary counters.

At no point does the transducer store the whole circuit in working memory. Hence r is computable in $O(\log n)$ workspace, i.e. it is a log-space reduction.

Summary. By construction, C_x faithfully simulates M on x : evaluating C_x on the given assignment yields 1 if and only if M accepts x , i.e. $x \in L$.

Since $L \in P$ was arbitrary, every language in P log-space reduces to CVP. Therefore CVP is P -hard under $\leq_L$.

CVP $\in P$ and every $L \in P$ log-space reduces to CVP, so CVP is P -complete under $\leq_L$.

□

Q10. **Bonus**

Prove that, for any given $e \in \mathbf{Tot}$, there exists $n \in \mathbb{N}$ such that $M_{M_e(n)} = M_n$.

Solution

Proof. Let $e \in \mathbf{Tot}$ be arbitrary, so M_e is a total computable function.

Step 1: Identifying a total computable function.

Define

$$t(n) := M_e(n).$$

Since $e \in \mathbf{Tot}$, the machine M_e halts on every input and always outputs a natural number. Therefore $t : \mathbb{N} \rightarrow \mathbb{N}$ is a *total computable* function.

Step 2: Applying the Recursion Theorem.

Kleene's Recursion Theorem. For every total computable function $t : \mathbb{N} \rightarrow \mathbb{N}$, there exists an index $n \in \mathbb{N}$ such that

$$M_n = M_{t(n)},$$

i.e. the machine at index n computes the same function as the machine at index $t(n)$.

Applying the Recursion Theorem to our total computable function t , we obtain an index $n \in \mathbb{N}$ satisfying

$$M_n = M_{t(n)}.$$

Step 3: Concluding.

Substituting $t(n) = M_e(n)$ into the fixed-point equality gives

$$M_n = M_{t(n)} = M_{M_e(n)}.$$

This is precisely the required equality.

For any $e \in \mathbf{Tot}$, the Recursion Theorem yields $n \in \mathbb{N}$ with $M_{M_e(n)} = M_n$.

□